Testbench Verification Report

Mixed-Width Asynchronous FIFO

Purpose, design and results of the six simulation testbenches for the VHDL core mixed_width_async_FIFO

Document	Testbench Verification Report
Design under test	`mixed_width_async_FIFO` (VHDL)
Author	Emir Uruç
Simulator	Vivado XSim
Testbenches	`tb_mixed_FIFO_1` to `tb_mixed_FIFO_6`
Configurations	8→32, 32→8 and 8→8
Result	Three directed tests as expected; three self-checking tests PASS

Contents

1. Introduction 3
1.1 Purpose and scope 3
1.2 Design under test 3
2. Verification approach 3
2.1 Risks and test mapping 3
2.2 Two kinds of testbench 4
2.3 Pass criteria 4
3. Test environment 4
3.1 Setup 4
3.2 Common testbench structure 4
3.3 Self-checking mechanism 5
4. Testbench descriptions 6
4.1 TB_1 – Narrow write, wide read (8→32) 6
4.2 TB_2 – Wide write, narrow read (32→8) 7
4.3 TB_3 – Full flag and overflow protection (32→8) 7
4.4 TB_4 – Streaming traffic (32→8) 8
4.5 TB_5 – Streaming traffic (8→32) 9
4.6 TB_6 – Equal-width regression (8→8) 10
5. Results 12
5.1 Summary 12
5.2 Simulation logs 12
5.3 Waveform observations 12
6. Coverage and gaps 12
6.1 What is covered 12
6.2 What is not covered 13
7. Conclusion 13
Appendix A. Running the testbenches 13

1. Introduction

1.1 Purpose and scope

This report describes the six testbenches written for the mixed-width asynchronous FIFO (`mixed_width_async_FIFO`). It explains why each testbench exists, what it is meant to prove, how the result is judged and what the simulation showed. It is the companion of the User Manual: the manual describes what the core does, this report describes how that behaviour was checked.

The scope is behavioural simulation of the RTL in Vivado XSim, with one data-width pair per testbench: 8→32, 32→8 and 8→8. Synthesis results, timing analysis and hardware tests are not part of this report.

1.2 Design under test

The core is a dual-clock FIFO whose write and read ports have different data widths. The storage consists of 2^`narrow_addr` locations of the narrower width (16 bytes by default). The wide port moves R = wide width / narrow width locations per access, so R = 4 for 8→32 and 32→8. The two width generics select one of three code branches inside the design, and the testbenches were chosen so that each branch is exercised.

Branch (generate label)	Condition	Data flow	Used by
`GEN_RD_WIDE`	`wr_width` < `rd_width`	write narrow, read wide (8→32)	TB_1, TB_5
`GEN_WR_WIDE`	`wr_width` > `rd_width`	write wide, read narrow (32→8)	TB_2, TB_3, TB_4
`GEN_EQUAL`	`wr_width` = `rd_width`	equal widths (8→8)	TB_6

2. Verification approach

2.1 Risks and test mapping

A mixed-width FIFO can fail in ways that a quick write-then-read test does not reveal. The table lists the risks that guided the design of the testbenches and shows which testbench addresses each of them.

Risk	What would go wrong	Addressed by
Wrong word order	Narrow words are packed into, or unpacked from, a wide word in the wrong order.	TB_1, TB_2 (waveform); TB_4, TB_5 (checked)
Wrong read threshold	A wide word is released before R narrow words are stored, or never released.	TB_1 (timing of `empty`); TB_5
Wrong write threshold	A wide write is accepted with fewer than R free locations, or refused too early.	TB_3; TB_4
Overflow	Unread data is overwritten while the FIFO is full.	TB_3; TB_4, TB_5, TB_6 (data checked)
Pointer wrap-around	Flags or addresses go wrong when a pointer wraps.	TB_4, TB_5, TB_6 (several wraps)
Equal-width regression	Shared pointer and synchronizer logic breaks the plain symmetric case.	TB_6

2.2 Two kinds of testbench

- **Directed tests (TB_1 to TB_3).** Fixed stimulus, evaluated by inspecting the waveform. They show the behaviour at specific points (packing, byte order, capacity limit) and are easy to reason about, but they contain no automatic check.
- **Streaming tests (TB_4 to TB_6).** A writer process and a reader process run independently of each other and react only to their own flag. 100 bytes flow through the FIFO and every received byte is compared with its expected value by an `assert`. They cover many more clock cycles than the directed tests and make the pointers wrap.

2.3 Pass criteria

Testbench type	Criterion
Directed (TB_1 to TB_3)	The expected values (data, byte order, flag timing, pointer values) are found in the waveform.
Streaming (TB_4 to TB_6)	No `assert` fires, and the simulation ends with the PASS message instead of the TIMEOUT message of the watchdog.

The checker itself was tested: in TB_4 the expected value was changed on purpose (to `i+2`) and the simulation stopped with FAIL at the first byte.

3. Test environment

3.1 Setup

Simulator	Vivado XSim, behavioural simulation of the RTL
Design file	`mixed_width_async_FIFO.vhd`
Testbenches	`tb_mixed_FIFO_1.vhd` to `tb_mixed_FIFO_6.vhd`
Write clock	10 ns period (100 MHz)
Read clock	7.5 ns period (133 MHz)
Reset	Both resets active for 100 ns, released at a falling edge of `clk_wr`
Generics	`narrow_addr` = 4 (16 locations), `N` = 3; data widths per testbench

3.2 Common testbench structure

All testbenches share the same structure. The directed tests keep reset and stimulus in one process (`P_STIMULI`); the streaming tests split them into separate processes.

Process	Task
`P_CLK_WR`, `P_CLK_RD`	Free-running write and read clocks.
`P_RESET`	Resets both domains (streaming tests; part of `P_STIMULI` in the directed tests).
`P_WRITE`	Drives `en_wr` and `din`. In the streaming tests it waits while `full` = '1' and offers the same word again until it is accepted.
`P_READ`	Drives `en_rd`. In the streaming tests it waits while `empty` = '1', then checks every byte or word, reports PASS and ends the simulation.
`P_WATCHDOG`	Stops the simulation with a TIMEOUT message after 5 µs if the test stalls (streaming tests).

Timing conventions

- Requests (`en_wr`, `en_rd`) and data are driven after the falling clock edge and removed after the next rising edge. The core therefore samples stable values at the rising edge and the testbench never races with the design.
- `full` and `empty` are examined at the falling edge. They only change at rising edges, so the value seen there is the value the core uses at the next rising edge.
- `dout` is registered, so it is checked one falling edge after the accepted read.

3.3 Self-checking mechanism

The streaming tests use consecutive numbers as test data, so the expected value is known at every moment. The writer sends the bytes 1, 2, 3, ... (packed least significant byte first when the write port is wide). The reader expects the same sequence: byte `i+1` for a narrow read, or the four bytes `4i+1` to `4i+4` concatenated for a wide read. A lost, repeated or reordered byte therefore fails the check at once.

> **Note on the log**
> After the last check the reader prints the PASS message and stops the simulation with `assert false ... severity failure`. This is why the XSim log ends with “Failure: SIM DONE”. It is the intended stop, not a design failure.

4. Testbench descriptions

Each description gives the configuration, the purpose, the stimulus and the expected result, followed by what the waveform showed. The waveform figures are full-length views; short pulses and bus values that are one clock apart appear compressed.

4.1 TB_1 – Narrow write, wide read (8→32)

Configuration	`wr_width` = 8, `rd_width` = 32 (R = 4), branch `GEN_RD_WIDE`
Check	Waveform inspection (directed test)
Simulation ends	451.25 ns

Purpose. Show that four consecutive 8-bit writes form one 32-bit word in the correct byte order, and that the wide word becomes readable only when all four bytes are stored.

Stimulus. After reset, four writes of 0x00, 0x01, 0x02 and 0x03 in consecutive clocks, a 100 ns pause, then one read.

Expected. `dout` = 0x03020100 after the read. `empty` is released about three read-clock cycles after the fourth write and returns to '1' after the read, because only one wide word was stored.

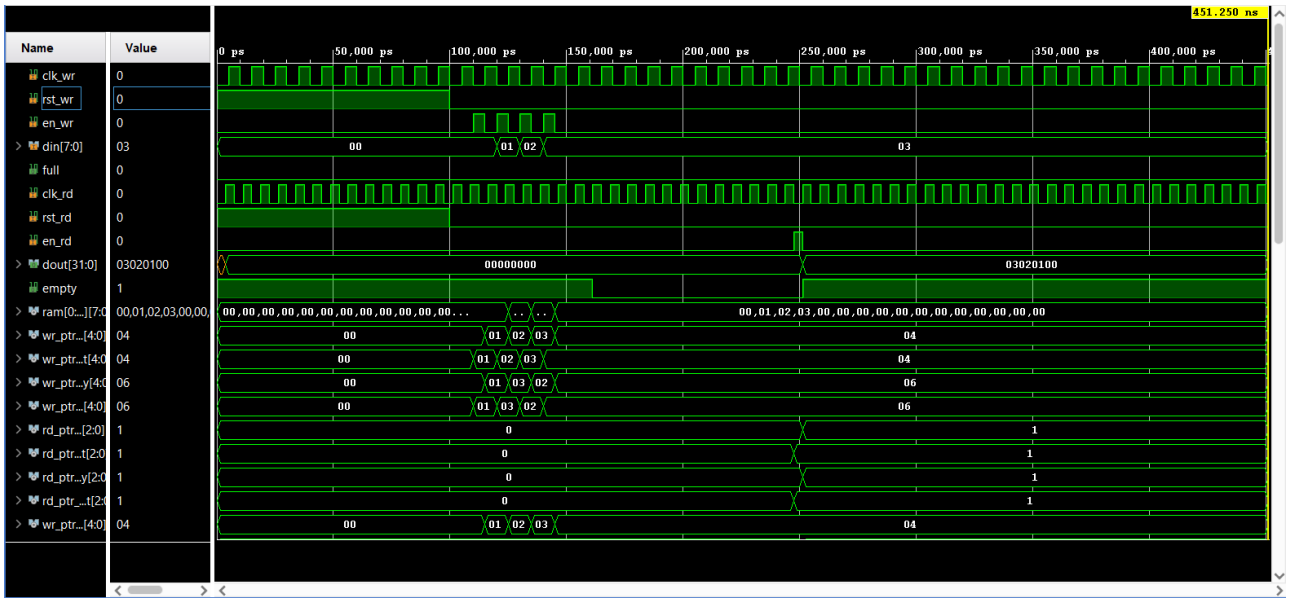


Figure 1 – TB_1: four byte writes, one 32-bit read.

Observed

- `ram` holds 00, 01, 02, 03 in locations 0 to 3; the write pointer reaches 4 (`wr_ptr_gray` = 06).
- `empty` falls about 16 ns after the fourth write (around 160 ns). That is three read-clock edges after the fourth write; a threshold of three bytes would have released it about 7 ns earlier.
- After the read `dout` = 0x03020100 and `rd_ptr_bin` = 1; `empty` returns to '1'.

4.2 TB_2 – Wide write, narrow read (32→8)

Configuration	`wr_width` = 32, `rd_width` = 8 (R = 4), branch `GEN_WR_WIDE`
Check	Waveform inspection (directed test)
Simulation ends	443.75 ns

Purpose. The counterpart of TB_1: show that one 32-bit write is stored in four consecutive locations and read out byte by byte, with the same byte-order convention.

Stimulus. After reset, one write of 0xABCD1234, a 100 ns pause, then four reads in consecutive clocks.

Expected. The reads return 0x34, 0x12, 0xCD, 0xAB (least significant byte first). `empty` is released about three read-clock cycles after the write and returns to '1' after the fourth read.

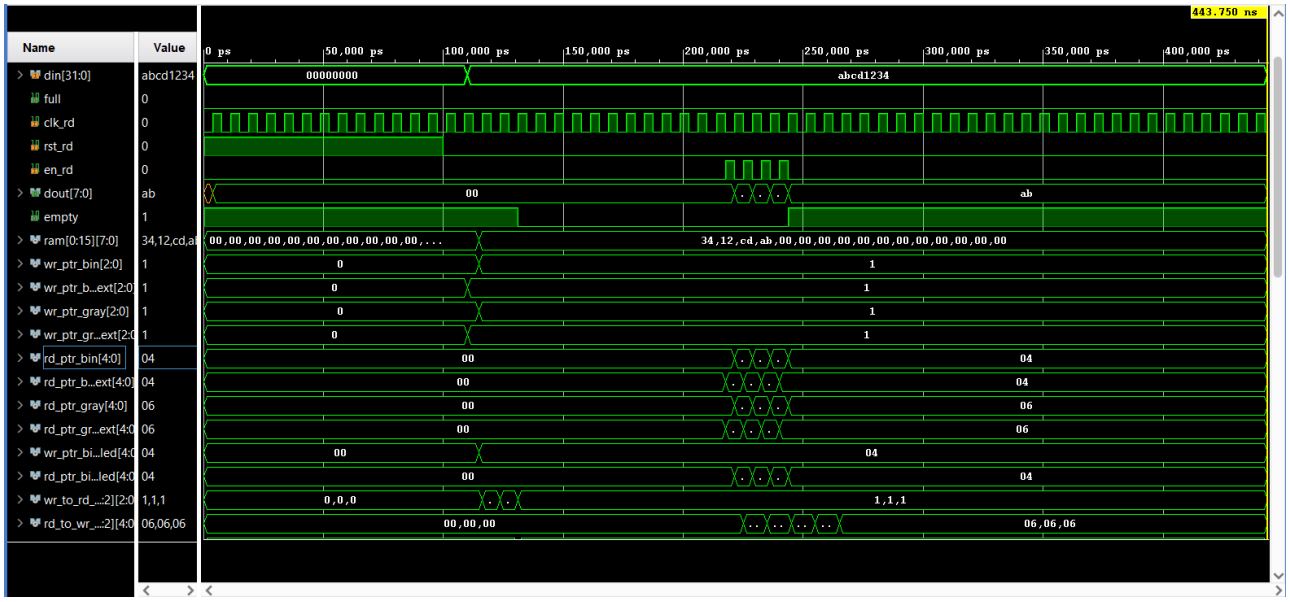


Figure 2 – TB_2: one 32-bit write, four byte reads.

Observed

- `ram` holds 34, 12, CD, AB in locations 0 to 3. The write pointer counts wide words, so `wr_ptr_bin` = 1 (`wr_ptr_gray` = 1).
- `rd_ptr_bin` = 04 and `rd_ptr_gray` = 06 after the four reads; `dout` ends at 0xAB.
- `empty` returns to '1' after the fourth read. In the full-length figure the four `dout` values are compressed; the sequence 0x34, 0x12, 0xCD, 0xAB was read from a zoomed view of the same run.

4.3 TB_3 – Full flag and overflow protection (32→8)

Configuration	`wr_width` = 32, `rd_width` = 8 (R = 4), branch `GEN_WR_WIDE`
Check	Waveform inspection (directed test)
Simulation ends	571.25 ns

Purpose. Show that the FIFO accepts exactly as many wide words as it can hold (four, for 16 locations) and refuses a further write without touching the stored data.

Stimulus. Five wide writes in consecutive clocks: 0x04030201, 0x08070605, 0x0C0B0A09, 0x100F0E0D and 0xFFFFFFFF, a 100 ns pause, then 16 byte reads. The fifth write is one wide word beyond the capacity and carries a recognisable value.

Expected. The first four writes are accepted and `full` rises with the fourth. The fifth write is presented while `full` = '1' and is ignored. The 16 reads return 0x01 to 0x10 and 0xFF never appears.

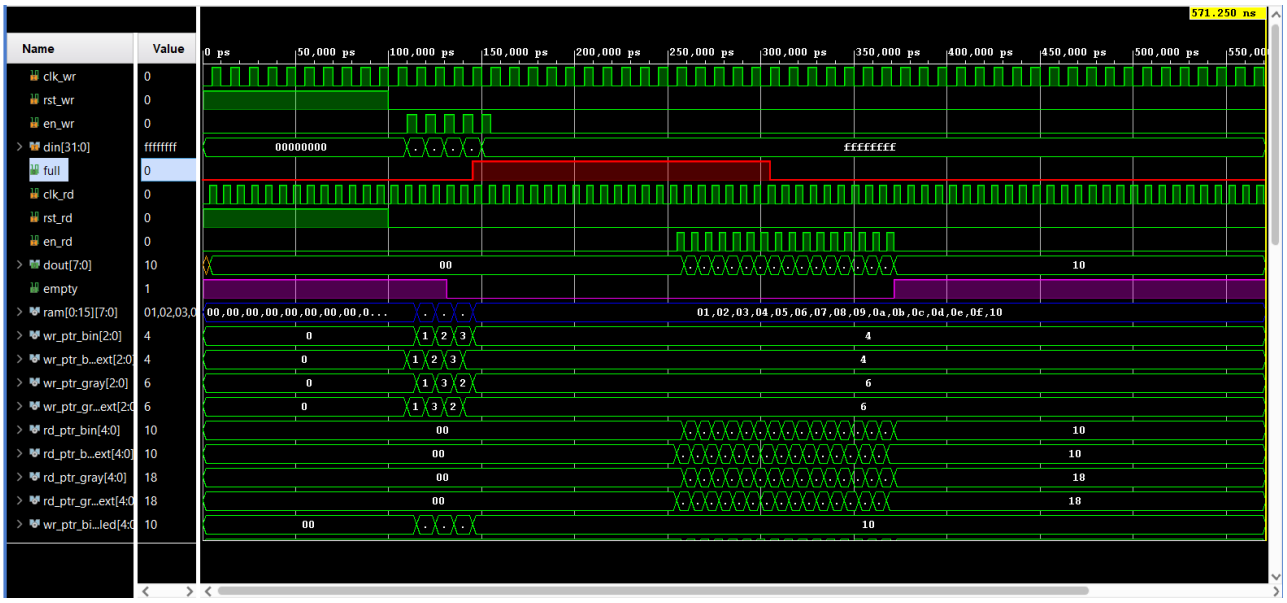


Figure 3 – TB_3: `full` (red) is already high when the fifth write is offered.

Observed

- `full` rises at about 145 ns, with the fourth write and before the fifth request. A threshold that is off by one wide word would show `wr_ptr_bin` stopping at 3 or reaching 5.
- `wr_ptr_bin` stops at 4 and `ram` holds 01, 02, ..., 0F, 10; the value 0xFF is not stored.
- The 16 reads end with `dout` = 0x10 and `rd_ptr_bin` = 0x10. `full` is released about 50 ns after the first read: four narrow reads free one wide word, plus the synchronizer delay of the read pointer.

4.4 TB_4 – Streaming traffic (32→8)

Configuration	`wr_width` = 32, `rd_width` = 8 (R = 4), branch `GEN_WR_WIDE`
Check	Self-checking: every byte compared; PASS message; 5 µs watchdog
Simulation ends	885 ns (PASS)

Purpose. Check correctness under continuous traffic with independent writer and reader, which is the situation the core sees in use, and let both pointers wrap.

Stimulus. The writer sends 25 wide words; word i contains the bytes 4i+1 to 4i+4. It waits whenever `full` = '1' and offers the same word again until it is accepted. The reader performs 100 narrow reads, one per read clock, waiting whenever `empty` = '1', and compares each byte with its expected value 1 to 100.

Expected. All 100 bytes arrive in order, nothing is overwritten, and the simulation ends with the PASS message before the watchdog fires.

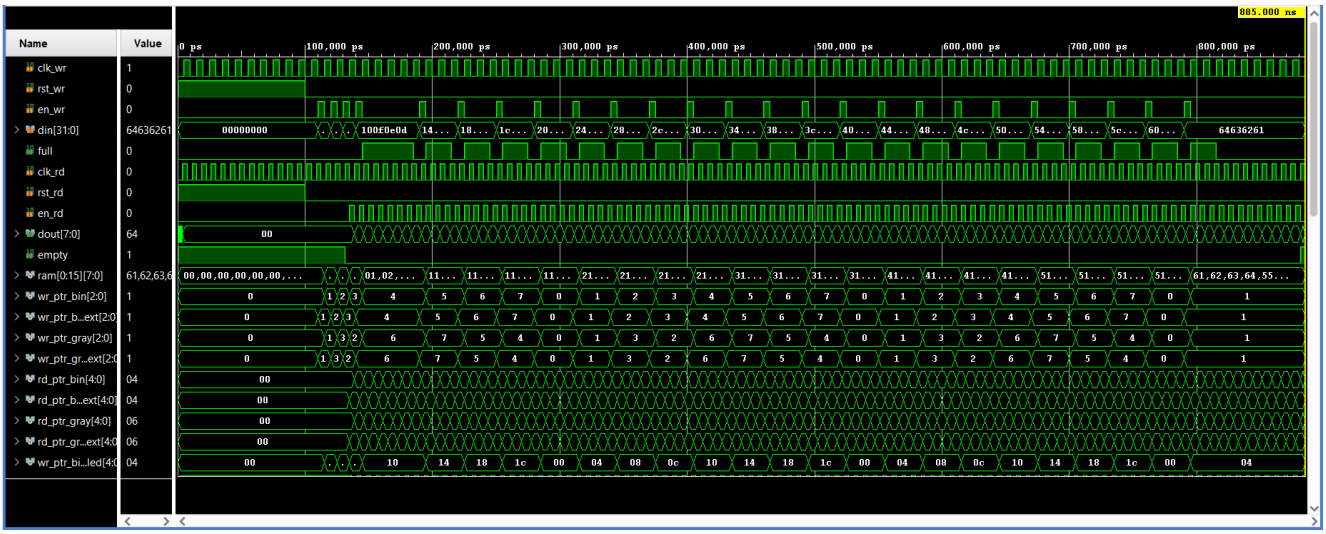


Figure 4 – TB_4: streaming traffic, 25 wide writes and 100 byte reads.

Observed

- The log reports PASS after 100 bytes at 885 ns; no assertion fired and the watchdog did not trigger.
- The writer pushes four bytes per write while the reader removes one byte per read clock, so the FIFO stays close to full. `full` rises and falls repeatedly, and the writer pauses each time until the reader has freed four locations. `empty` is only active at the start and at the end.
- The write pointer (3 bits) wraps several times (25 writes, final value 1); the read pointer (5 bits) wraps three times (100 reads, final value 04). The Gray pointers change one bit per step, also at the wrap.
- The final `ram` content is 61, 62, 63, 64, 55, ..., 60: the last 16 bytes written.

4.5 TB_5 – Streaming traffic (8→32)

Configuration	`wr_width` = 8, `rd_width` = 32 (R = 4), branch `GEN_RD_WIDE`
Check	Self-checking: every word compared; PASS message; 5 µs watchdog
Simulation ends	1852.5 ns (PASS)

Purpose. The same stress as TB_4 in the opposite direction. This is also the only test that fills the FIFO through the narrow write port, so it exercises the `full` logic of the `GEN_RD_WIDE` branch.

Stimulus. The writer sends 100 single bytes (values 1 to 100) and waits while `full` = '1'. The reader performs 25 wide reads, waiting while `empty` = '1', and checks that each word equals the concatenation of the next four bytes. After every read it pauses for 60 ns.

Why the pause. A wide read removes four bytes per read clock, much faster than the writer adds them (one byte per 10 ns). Without the pause the FIFO would stay almost empty and `full` would never be reached. The pause makes the reader the slower side, so the FIFO fills and `full` cycles.

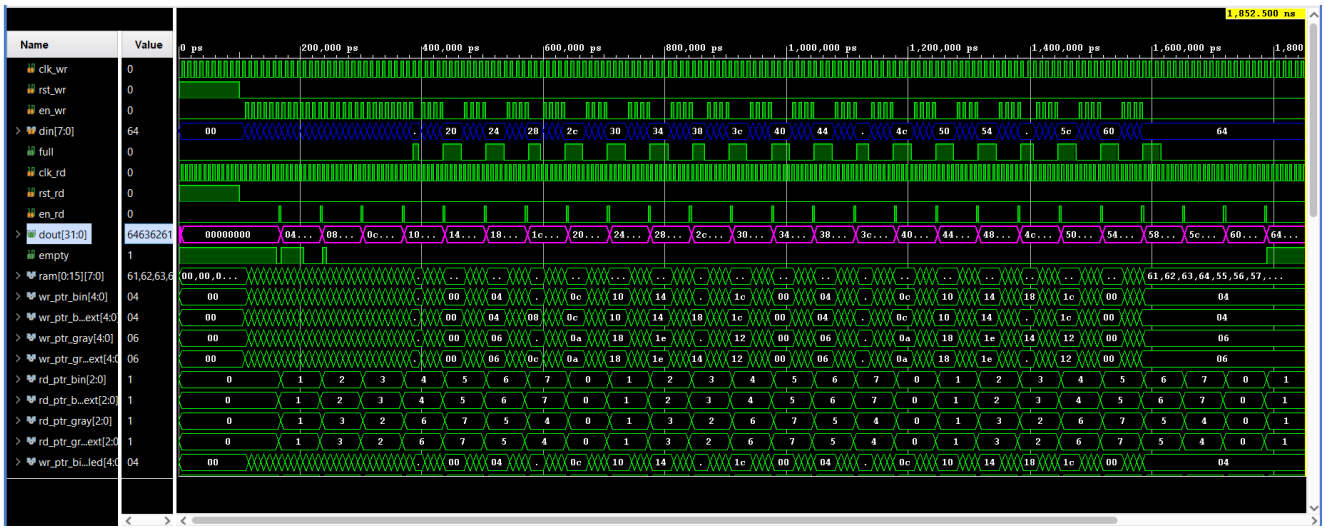


Figure 5 – TB_5: 100 byte writes and 25 word reads with a pause after each read.

Observed

- The log reports PASS at 1852.5 ns; no assertion fired and the watchdog did not trigger.
- `full` rises and falls repeatedly. The writer idles while `full` = '1', so the write side runs ahead and waits for the reader.
- The write pointer (5 bits) wraps three times (final value 04); the read pointer (3 bits) wraps three times (25 reads, final value 1). `rd_ptr_gray` runs 1, 3, 2, 6, 7, 5, 4, 0, one bit per step, including the wrap.
- `wr_ptr_bin_fe_scaled` (the write pointer in narrow-word units) runs 00, 04, 08, 0C, ..., 1C, 00, so the scaled value wraps with the pointer.

4.6 TB_6 – Equal-width regression (8→8)

Configuration	`wr_width` = 8, `rd_width` = 8 (R = 1), branch `GEN_EQUAL`
Check	Self-checking: every byte compared; PASS message; 5 µs watchdog
Simulation ends	2385 ns (PASS)

Purpose. Confirm that the equal-width case still works as a plain asynchronous FIFO after the mixed-width logic was changed. This branch uses its own Gray-code comparison for `full` and `empty` (no scaling), but it shares the pointer generation, the synchronizers and the Gray conversion with the mixed branches, and those parts were reworked while the mixed-width logic was corrected.

Stimulus. The same streaming pattern as TB_4 and TB_5: 100 bytes written, 100 bytes read and checked. The reader pauses 10 ns after every read (about 22.5 ns per byte against 10 ns per byte for the writer), so the FIFO fills and `full` cycles.

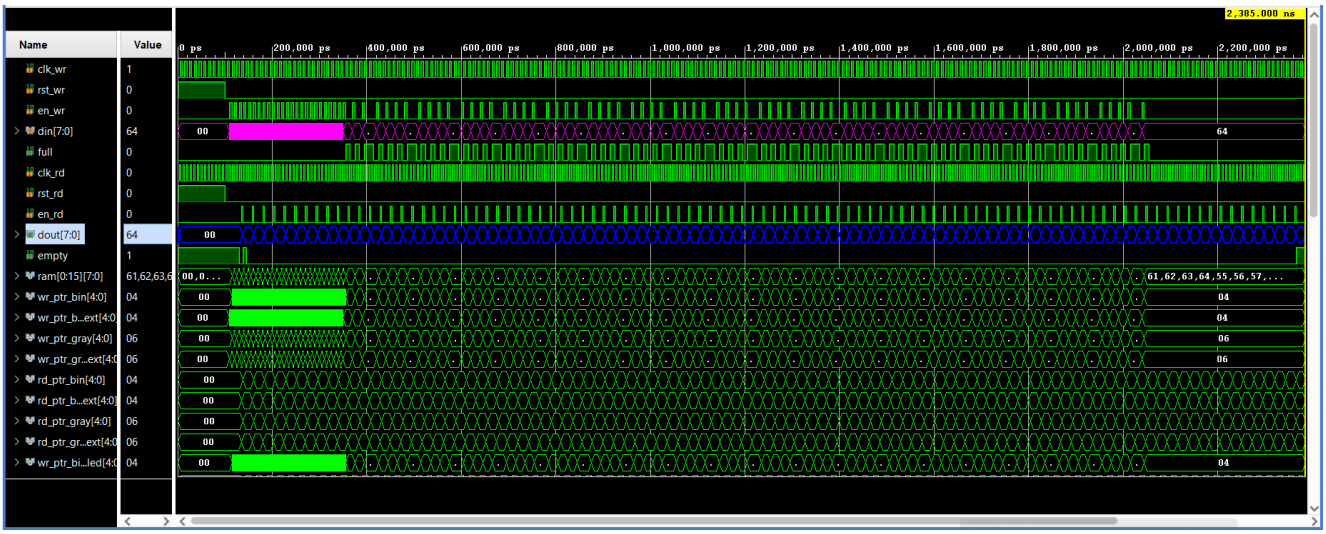


Figure 6 – TB_6: streaming traffic with equal widths.

Observed

- The log reports PASS at 2385 ns; no assertion fired and the watchdog did not trigger.
- Both pointers are 5 bits wide and wrap three times; both end at 04 and both Gray pointers at 06.
- `full` rises and falls repeatedly; `empty` is active only at the start and at the end.

5. Results

5.1 Summary

TB	Widths	Branch	Check	Ends at	Result
1	8→32	GEN_RD_WIDE	Waveform	451.25 ns	As expected
2	32→8	GEN_WR_WIDE	Waveform	443.75 ns	As expected
3	32→8	GEN_WR_WIDE	Waveform	571.25 ns	As expected
4	32→8	GEN_WR_WIDE	Self-checking	885 ns	PASS
5	8→32	GEN_RD_WIDE	Self-checking	1852.5 ns	PASS
6	8→8	GEN_EQUAL	Self-checking	2385 ns	PASS

“PASS” is used only for the three self-checking testbenches. The directed tests have no automatic check; their result is the agreement of the waveform with the expected values given in section 4.

5.2 Simulation logs

The log of each self-checking testbench contains the following lines (file paths removed):

```
tb_mixed_FIFO_4 :  Note: PASS:  100 bytes were read correctly
                   Time: 885 ns        Failure: SIM DONE
tb_mixed_FIFO_5 :  Note: PASS:  100 bytes were read correctly
                   Time: 1852500 ps    Failure: SIM DONE
tb_mixed_FIFO_6 :  Note: PASS:  100 bytes were read correctly
                   Time: 2385 ns       Failure: SIM DONE
```

5.3 Waveform observations

- **One bit per step.** In every run the Gray pointers change a single bit per step, including at the wrap-around (for example `rd_ptr_gray` in TB_5).
- **Scaling after the crossing.** The wide-side pointer is multiplied by R only after the synchronizer (for example 00, 04, 08, 0C, 10, ..., 1C, 00), so the signal that crosses the clock boundary stays a plain Gray counter.
- **Skipped synchronizer values.** The synchronized pointer of the other domain sometimes skips values, because the clock of the receiving side is slower than the other side. This is expected. The flags are conservative: a flag is released a few cycles after the other side has made room or data available, never earlier.
- **No overflow or data loss.** Overwriting unread data would corrupt bytes and the checkers of TB_4 to TB_6 would fail; all three passed. TB_3 shows the refused write directly.

6. Coverage and gaps

6.1 What is covered

Area	Evidence
Narrow write, wide read data path	TB_1 (directed), TB_5 (streaming)
Wide write, narrow read data path	TB_2, TB_3 (directed), TB_4 (streaming)
Byte order (first narrow word = least significant)	TB_1, TB_2 by waveform; TB_4, TB_5 checked automatically
Read threshold of the wide read (R words)	TB_1 (timing of `empty`); TB_5

Area	Evidence
Write threshold of the wide write (R free locations)	TB_3 (exact capacity), TB_4
Overflow protection	TB_3 (write refused while full); TB_4 to TB_6 (writer waits, data intact)
Pointer wrap-around, both pointers	TB_4, TB_5, TB_6
Gray pointer single-bit change	All testbenches (waveform), including the wrap
Equal-width branch	TB_6

6.2 What is not covered

- **Empty boundary during streaming.** In TB_4 to TB_6 the reader is the slow side, so `empty` is active only at start-up and at the end. A fast reader that is repeatedly starved was not simulated.
- **Requests while a flag is set.** A write while `full` = '1' is tested only for the wide write (TB_3). A read while `empty` = '1' was never issued, and a write while `full` = '1' was not tested for the narrow write port.
- **Random timing.** Enables are not randomized, and the clock ratio is fixed at 4:3 (edge alignment repeats every 30 ns).
- **Other parameters.** Other width ratios (R = 2, 8), other `narrow_addr` and `N` values.
- **Reset behaviour.** Reset during operation and reset of one side only.
- **Other verification levels.** Post-synthesis simulation, formal CDC analysis and hardware tests.

7. Conclusion

All six testbenches ran to completion. The three directed tests show the expected packing and unpacking, the byte order and the capacity limit of the wide write. The three streaming tests passed their automatic checks for the three data-width configurations: 100 bytes arrived in the correct order, the pointers wrapped several times, and `full` cycled repeatedly without any data being overwritten.

The remaining gaps are listed in section 6.2. They concern the empty boundary during streaming, random timing, other width ratios and reset behaviour, and not the behaviour that the six testbenches cover.

Appendix A. Running the testbenches

- Add `mixed_width_async_FIFO.vhd` as a design source and the testbench as a simulation source; set the testbench as the simulation top.
- Start the behavioural simulation. Vivado runs 1000 ns by default, which is enough for TB_1 to TB_4. TB_5 (1852.5 ns) and TB_6 (2385 ns) need **Run All** or `run all` in the Tcl console.
- The streaming testbenches stop themselves with the line “Failure: SIM DONE”. If neither PASS nor a FAIL message appears, the test stalled; run to the end and check the TIMEOUT message of the watchdog.